Graph Algorithms in C++

A Beginner's Manual — Simplest Code, Clear Logic

What is a Graph?

A graph is just a set of nodes (vertices) connected by edges. Think of cities connected by roads.

Two types you'll use most:

- Undirected: edge goes both ways (road between two cities)
- Weighted: each edge has a number (distance, cost, etc.)

Three ways to store a graph:

- Edge List: just store each edge as (u, v, weight)
- Adjacency Matrix: 2D array, matrix[u][v] = weight
- Adjacency List: for each node, store list of its neighbors

Q1 — Weighted Graph Stats

Read a weighted graph. Print: number of nodes, edges, total weight, min weight, max weight.

The Idea

Just read edges one by one. Keep running sum, track min and max. That's it.

Algorithm (4 steps)

1. Read N (nodes) and M (edges)
2. Read each edge: u v w
3. Add w to totalWeight, update min/max
4. Print the 5 values

Code

```
#include <iostream>
#include <climits>    // for INT_MAX, INT_MIN
using namespace std;

int main() {
    int N, M;
    cin >> N >> M;      // read nodes and edges

    int totalWeight = 0;
    int minEdge = INT_MAX;    // start very high
    int maxEdge = INT_MIN;    // start very low
```

```
    for (int i = 0; i < M; i++) {
        int u, v, w;
        cin >> u >> v >> w;    // read one edge

        totalWeight += w;
        if (w < minEdge) minEdge = w;   // update min
        if (w > maxEdge) maxEdge = w;   // update max
    }

    cout << "Nodes: "        << N           << "\n";
    cout << "Edges: "        << M           << "\n";
    cout << "Total Weight: " << totalWeight << "\n";
    cout << "Min Edge: "     << minEdge     << "\n";
    cout << "Max Edge: "     << maxEdge     << "\n";
    return 0;
}
```

Example

INPUT	OUTPUT
4 5 0 1 1 0 2 3 1 2 1 1 3 4 2 3 2	Nodes: 4 Edges: 5 Total Weight: 11 Min Edge: 1 Max Edge: 4

Weights are: 1, 3, 1, 4, 2 → sum=11, min=1, max=4. Simple loop.

Q2 — Union-Find (Disjoint Set Union)

Track which nodes are 'connected'. Support two operations: unite two nodes, query if two nodes share a group.

The Idea

Imagine each node as a village. Each village has a 'chief' (representative). Two villages are connected if they share a chief.
union(a, b) → merge villages of a and b (make them share a chief)
query(a, b) → Yes if find(a) == find(b), else No

> **Key Trick: Path Compression**
> When we call find(x), make x point directly to the root. Next time it's instant.

Code

```
#include <iostream>
#include <string>
using namespace std;
```

```
int parent[100005];

// Find root of x (with path compression)
int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);  // shortcut path
    return parent[x];
}

// Merge groups of a and b
void unite(int a, int b) {
    parent[find(a)] = find(b);
}

int main() {
    int N, T;
    cin >> N >> T;

    // Initially each node is its own parent
    for (int i = 0; i < N; i++) parent[i] = i;

    for (int i = 0; i < T; i++) {
        string op;
        int a, b;
        cin >> op >> a >> b;

        if (op == "union") {
            unite(a, b);
        } else {  // "query"
            if (find(a) == find(b))
                cout << "query " << a << " " << b << " = Yes\n";
            else
                cout << "query " << a << " " << b << " = No\n";
        }
    }

    // Count distinct roots = number of components
    int components = 0;
    for (int i = 0; i < N; i++)
        if (find(i) == i) components++;  // i is a root

    cout << "Components: " << components << "\n";
    return 0;
}
```

Example

INPUT

```
5 6
union 0 1
union 2 3
query 0 1
query 0 2
union 1 3
query 0 3
```

OUTPUT

```
query 0 1 = Yes
query 0 2 = No
query 0 3 = Yes
Components: 2
```

Trace walkthrough

```
Start:  parent = [0,1,2,3,4]   (each alone)
union(0,1) → parent[0]=1   {0,1} {2} {3} {4}
union(2,3) → parent[2]=3   {0,1} {2,3} {4}
query(0,1) → find(0)=1, find(1)=1 → Yes
query(0,2) → find(0)=1, find(2)=3 → No
union(1,3) → parent[1]=3   {0,1,2,3} {4}
query(0,3) → find(0)=3, find(3)=3 → Yes
Components: roots are 3 and 4 → 2
```

Q3 — Kruskal's MST

Find Minimum Spanning Tree: connect all nodes using the cheapest possible edges, no cycles.

The Idea

Greedy: always pick the cheapest edge that doesn't form a cycle.
How to detect cycles? Use Union-Find! If two endpoints already share a group → adding this edge creates a cycle → skip it.

Algorithm (3 steps)

1. Sort all edges by weight (smallest first)
2. For each edge (u,v,w): if find(u) != find(v) → add it, unite(u,v)
3. MST complete when N-1 edges added (Connected: Yes)

Code

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int parent[100005];

int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]);
    return parent[x];
}
void unite(int a, int b) {
    parent[find(a)] = find(b);
}

int main() {
    int N, M;
    cin >> N >> M;

    // Store edges as {weight, u, v} for easy sorting
    vector<tuple<int,int,int>> edges(M);
    for (auto& [w, u, v] : edges) cin >> u >> v >> w;
```

```
    // Sort by weight (ties broken by u then v)
    sort(edges.begin(), edges.end());

    for (int i = 0; i < N; i++) parent[i] = i;

    int mstWeight = 0, cables = 0;

    for (auto& [w, u, v] : edges) {
        if (find(u) != find(v)) {   // no cycle
            unite(u, v);
            mstWeight += w;
            cables++;
        }
    }

    cout << "MST Weight: " << mstWeight << "\n";
    cout << "Cables: "     << cables    << "\n";
    cout << "Connected: "  << (cables == N-1 ? "Yes" : "No") << "\n";
    return 0;
}
```

Example

INPUT
```
4 5
0 1 1
0 2 3
1 2 1
1 3 4
2 3 2
```

OUTPUT
```
MST Weight: 4
Cables: 3
Connected: Yes
```

Trace walkthrough
```
Sorted edges: (1,0,1) (1,1,2) (2,2,3) (3,0,2) (4,1,3)
Edge 0-1 w=1: diff groups → ADD. cables=1, w=1
Edge 1-2 w=1: diff groups → ADD. cables=2, w=2
Edge 2-3 w=2: diff groups → ADD. cables=3, w=4
Edge 0-2 w=3: same group → SKIP (cycle!)
Edge 1-3 w=4: same group → SKIP (cycle!)
cables=3 = N-1=3 → Connected: Yes
```

Q4 — Prim's MST

Same goal as Kruskal but different approach: grow one tree from node 0, always picking the cheapest edge to a new node.

The Idea

Think of it like growing a plant. Start at node 0. Look at all edges from your current tree. Pick the cheapest one leading to an unvisited node. Add that node. Repeat.
Use a min-heap (priority_queue) to always get the cheapest edge fast.

Algorithm
1. Build adjacency list from edges
2. Push (0, node 0) into min-heap. Mark all unvisited.
3. Pop cheapest (cost, node). If already visited, skip.
4. Mark visited. Add cost to MST. Add to build order.
5. Push all unvisited neighbors into heap. Repeat step 3.

Code

```
#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>
using namespace std;

int main() {
    int N, M;
    cin >> N >> M;

    // adj[u] = list of {v, weight}
    vector<vector<pair<int,int>>> adj(N);

    for (int i = 0; i < M; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});  // undirected
    }

    // Sort adjacency lists for deterministic order
    for (int i = 0; i < N; i++)
        sort(adj[i].begin(), adj[i].end());

    // Min-heap: {cost, node}
    // priority_queue is max-heap by default, use negative or greater<>
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
    vector<bool> visited(N, false);

    pq.push({0, 0});  // start from node 0, cost 0
    int mstWeight = 0;
    vector<int> buildOrder;

    while (!pq.empty()) {
        auto [cost, u] = pq.top(); pq.pop();

        if (visited[u]) continue;  // already in tree
        visited[u] = true;
        mstWeight += cost;
        buildOrder.push_back(u);

        for (auto [v, w] : adj[u]) {
            if (!visited[v])
                pq.push({w, v});
        }
    }
```

```
    cout << "MST Weight: " << mstWeight << "\n";
    cout << "Build Order: ";
    for (int i = 0; i < buildOrder.size(); i++) {
        if (i) cout << " ";
        cout << buildOrder[i];
    }
    cout << "\n";
    return 0;
}
```

Example

INPUT	OUTPUT
4 5 0 1 1 0 2 3 1 2 1 1 3 4 2 3 2	MST Weight: 4 Build Order: 0 1 2 3

Trace walkthrough

```
Heap: [(0,0)]
Pop (0,0): visit 0. Push (1,1),(3,2). Order=[0]
Pop (1,1): visit 1. Push (1,2),(4,3). Order=[0,1]
Pop (1,2): visit 2. Push (2,3). Order=[0,1,2]
Pop (2,3): visit 3. Order=[0,1,2,3]
MST Weight = 0+1+1+2 = 4
```

Q5 — MST Weight & Bottleneck

Kruskal's MST + find the heaviest edge used + count connected components.

The Idea

Same as Kruskal. While picking edges, also track the max weight used (that's the bottleneck). Components = N minus edges chosen.

Code

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int parent[100005];

int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]);
    return parent[x];
}
```

```
void unite(int a, int b) {
    parent[find(a)] = find(b);
}

int main() {
    int N, M;
    cin >> N >> M;

    vector<tuple<int,int,int>> edges(M);
    for (auto& [w, u, v] : edges) cin >> u >> v >> w;
    sort(edges.begin(), edges.end());

    for (int i = 0; i < N; i++) parent[i] = i;

    int mstWeight = 0, chosen = 0, bottleneck = 0;

    for (auto& [w, u, v] : edges) {
        if (find(u) != find(v)) {
            unite(u, v);
            mstWeight += w;
            chosen++;
            if (w > bottleneck) bottleneck = w;  // track max
        }
    }

    int components = N - chosen;  // key formula

    cout << "MST Weight: "  << mstWeight  << "\n";
    cout << "Bottleneck: "  << bottleneck << "\n";
    cout << "Components: "  << components << "\n";
    return 0;
}
```

Example

INPUT
```
5 6
0 1 1
0 2 3
1 2 1
1 3 4
2 3 2
3 4 5
```

OUTPUT
```
MST Weight: 9
Bottleneck: 5
Components: 1
```

Why Components = N - chosen?
If N=5 nodes and we pick 4 edges with no cycle → one connected tree (5-4=1 component).
If disconnected graph: we pick fewer edges → more components remain.

Q6 — Max vs Min Spanning Tree

Run Kruskal twice: once for minimum (sort ascending), once for maximum (sort descending). Print both weights and their difference.

The Idea

Kruskal picks cheapest edges → Min Spanning Tree.
Same algorithm but sort edges heaviest-first → Max Spanning Tree.
Same union-find core. Just change sort order.

Code

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int parent[100005];

int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]);
    return parent[x];
}
void unite(int a, int b) {
    parent[find(a)] = find(b);
}

// Run Kruskal on given edge order, return MST weight
int kruskal(int N, vector<tuple<int,int,int>>& edges) {
    for (int i = 0; i < N; i++) parent[i] = i;  // reset
    int total = 0;
    for (auto& [w, u, v] : edges) {
        if (find(u) != find(v)) {
            unite(u, v);
            total += w;
        }
    }
    return total;
}

int main() {
    int N, M;
    cin >> N >> M;

    vector<tuple<int,int,int>> edges(M);
    for (auto& [w, u, v] : edges) cin >> u >> v >> w;

    // Min spanning tree: sort ascending
    sort(edges.begin(), edges.end());
    int minW = kruskal(N, edges);

    // Max spanning tree: sort descending (reverse)
    sort(edges.rbegin(), edges.rend());
    int maxW = kruskal(N, edges);

    cout << "Max Spanning Weight: " << maxW        << "\n";
    cout << "Min Spanning Weight: " << minW        << "\n";
    cout << "Difference: "          << maxW - minW << "\n";
    return 0;
}
```

Example

INPUT
```
4 5
0 1 1
0 2 3
1 2 1
1 3 4
2 3 2
```

OUTPUT
```
Max Spanning Weight: 9
Min Spanning Weight: 4
Difference: 5
```

Key insight
Max tree picks: edges 4, 3, 2 = weight 9
Min tree picks: edges 1, 1, 2 = weight 4
Difference = 9 - 4 = 5

Quick Reference Cheat Sheet

Question	Key Idea	Data Structure	Complexity
Q1 Graph Stats	Loop, sum, min/max	Edge list	O(M)
Q2 Union-Find	Find root, merge groups	parent[] array	O(α N) ≈ O(1)
Q3 Kruskal MST	Sort + Union-Find	Edge list + UF	O(M log M)
Q4 Prim MST	Grow from node 0, min-heap	Adj list + PQ	O(M log N)
Q5 Bottleneck	Kruskal + track max	Edge list + UF	O(M log M)
Q6 Max vs Min MST	Kruskal twice (asc/desc)	Edge list + UF x2	O(M log M)

Common Mistakes to Avoid

Mistake	Fix
Forgetting to init parent[i]=i	Always reset before each Kruskal run
Not sorting edges before Kruskal	sort() is what makes greedy work
Counting edges twice for undirected	Each edge weight counted once in Q1
Using max-heap for Prim's	Add greater<> to get min-heap
Forgetting path compression in find()	Without it, find() is slow on big inputs